OMNIbus Lab — Student Schedule

Duration: 2.5 days
Format: Instructor-led, hands-on lab
Daily hours: 8:30 AM – 5:00 PM (7.5 hours per day, ~20 hours total)

> This schedule is a guide, not a strict contract. Some sections will take longer depending on your familiarity with Linux, SQL, and networking concepts. If you fall behind, the instructor will help you catch up. Do not skip sections — each one builds directly on the previous.

Day 1 — Foundation and Event Management

8:30 AM — Welcome and Environment Check *(30 min)*

- Introductions and course overview
- Instructor distributes lab server IP addresses and credentials
- Verify SSH access to assigned lab server
- Verify OMNIbus tools are installed on Windows workstation (Administrator client, Event List, MIB Manager)
- Open the lab guide index: `omnibus-lab-guide.md`

9:00 AM — Section 1: OMNIbus Fundamentals *(30 min)*

Reading and discussion — no hands-on tasks

- 1.1 OMNIbus overview and the event pipeline
- 1.2 Core architecture and components
- 1.3 Event lifecycle — ingestion, deduplication, correlation
- 1.4 Key processes and terminology
- 1.5 Lab environment and assumptions

> **Key takeaway:** Internalize the pipeline — source → probe → ObjectServer → operator. Every hands-on task in the rest of the course is a piece of this model.

9:30 AM — Section 2.1: Environment Access and Tool Validation *(60 min)*

Hands-on — SSH, server startup, Windows client tools, Web GUI

- Set environment variables (`NCHOME`, `OMNIHOME`, `PATH`)
- Start the ObjectServer via Process Agent (`nco_pad`)
- Verify with `nco_ping EVENTS`
- Check / start JazzSM Web GUI server
- Configure Servers Editor on Windows workstation
- Connect via native Event List client and Netcool/OMNIbus Administrator
- Log in to Web GUI at `https://<your-server>:16311/ibm/console`

> **Checkpoint:** All four connections working — SSH, Event List, Administrator, Web GUI.

10:30 AM — Break *(15 min)*

10:45 AM — Section 2.2: Basic Event Interaction *(75 min)*

Hands-on — Web GUI event operations, filters, views, Simnet

- Navigate the event list; understand severity levels 0–5

- Prioritize, acknowledge, and add journal entries to events
- Use filters (Default, Escalated) and views (Default, Self-Monitoring)
- Start the Simnet probe; observe live deduplication via `FirstOccurrence`, `LastOccurrence`, `Count`

 Checkpoint: You can acknowledge an event, add a journal entry, escalate an alert, switch filters and views, and see Simnet events deduplicating in real time.

12:00 PM — Lunch *(60 min)*

1:00 PM — Section 3.1: The ObjectServer Users Table *(60 min)*

Hands-on — Netcool/OMNIbus Administrator + Web GUI + nco_sql

- Review default users and groups in Netcool/OMNIbus Administrator
- Create the `netcool` user; assign to `Netcool_OMNIbus_User` group
- Assign Web GUI roles (`ncw_user`, `netcool_rw`, `operator`) in the Web GUI
- Verify login as `netcool` user
- Use `nco_sql` to create `jdoe` and assign group memberships via SQL
- Understand the two-layer security model: ObjectServer groups vs. Web GUI roles

 Checkpoint: `netcool` user can log in to Web GUI and access events. `jdoe` created via `nco_sql` with correct group membership.

2:00 PM — Section 3.2: The alerts.status Table and Important Event Fields *(45 min)*

Hands-on — Netcool/OMNIbus Administrator + nco_sql

- Browse the `alerts.status` schema in Netcool/OMNIbus Administrator
- Understand key fields: `Node`, `Summary`, `AlertGroup`, `AlertKey`, `Severity`, `Type`, `Identifier`
- Add `CustomerID` column via the Administrator GUI
- Add `CircuitID` column via `nco_sql`
- Verify both new columns appear in the Web GUI event details

 Checkpoint: Both `CustomerID` and `CircuitID` exist in `alerts.status`.

2:45 PM — Break *(15 min)*

3:00 PM — Section 3.3: Reviewing ObjectServer Triggers *(45 min)*

Read and inspect — Netcool/OMNIbus Administrator, no configuration changes

- Open and read the `deduplication` trigger (pre-insert on reinsert)
- Open and read the `generic_clear` trigger (temporal, every 5 seconds)
- Open and read the `expire` trigger (temporal, time-based auto-clear)
- Understand how `old.<column>` vs. `new.<column>` work in trigger logic

Key takeaway: These three triggers directly govern the behavior of everything you build in Sections 4 and 5. Know them.

3:45 PM — Section 3.4: ObjectServer Properties *(45 min)*

Hands-on — Netcool/OMNIbus Administrator + nco_sql + curl

- Review the properties list; understand Editable vs. Immediate flags

- Enable the REST API: set `NRestOS.Enable`, `NHttpd.EnableHTTP`, `NHttpd.ListeningPort`
- Shut down the ObjectServer via `nco_sql` (`alter system shutdown`)
- Wait for Process Agent to restart it; verify with `nco_ping`
- Test the REST API with `curl`

 Checkpoint: `curl` against `http://localhost:8888/objectserver/restapi/alerts/status` returns JSON alert data.

4:30 PM — Day 1 Wrap-Up *(30 min)*

- Instructor recap: what was covered, what concepts to let settle overnight
- Student questions
- Leave the ObjectServer and JazzSM running for Day 2

Day 2 — Probes, Rules, and SNMP

8:30 AM — Day 2 Warm-Up *(15 min)*

- Verify SSH session and environment variables are set
- Confirm `nco_ping EVENTS` succeeds and Web GUI is accessible
- Brief review of Day 1 key concepts

8:45 AM — Section 4.0: Lab Setup — Filter and View Configuration *(30 min)*

Hands-on — Web GUI filter and view builder

- Create the `SyslogdProbe` filter (`Manager like 'MyProbe'`)
- Create the `SyslogdView` view with 15 fields in specified order
- Confirm filter and view are independent controls

 Checkpoint: `SyslogdProbe` filter and `SyslogdView` view active in Web GUI — event list is empty (expected at this stage).

9:15 AM — Section 4.1: Probe Overview and Architecture *(20 min)*

Hands-on — SSH, read-only file inspection

- List available probes under `$OMNIHOME/probes/`
- Review `syslogd.props` and `syslogd.rules` file structure
- Identify key properties: `Server`, `MessageLevel`, `RawCapture`, `RulesFile`

9:35 AM — Section 4.2: Syslogd Probe Configuration and Testing *(45 min)*

Hands-on — SSH, properties file edit, probe start, test event

- Set `Server : 'EVENTS'` in `syslogd.props`
- Start the probe; verify process is running
- Send a test event with `logger -n 127.0.0.1 "Test event..."`
- Enable `MessageLevel : 'debug'` and `RawCapture : 1`; restart probe
- Inspect the probe log and raw capture file; identify $Raw and positional tokens

 Checkpoint: Test event appears in Web GUI. $Raw token visible in capture file.

10:20 AM — Break *(15 min)*

10:35 AM — Section 4.3: Probe Rules by Example *(2 hrs 30 min)*

Hands-on — SSH, iterative rules development

For each subsection: edit rules file → syntax check → restart probe → send test event → verify in Web GUI.

- **4.3.1** *(20 min)* — Create `syslogd-training.rules`; add ProbeWatch stanza and basic `@Manager`, `@AlertGroup` assignments
- **4.3.2** *(15 min)* — Add `extract()` to populate `@EventId` from $Raw; understand greedy vs. non-greedy matching
- **4.3.3** *(20 min)* — Add `switch/case` on `@EventId`; map `LinkDown` fields; observe `default` case
- **4.3.4** *(20 min)* — Add `@Identifier` to `LinkDown`; send repeated events; observe deduplication
- **4.3.5** *(20 min)* — Add `LinkUp` case (`@Type = 2`); send LinkDown then LinkUp; observe `generic_clear` auto-clear
- **4.3.6** *(25 min)* — Add `LinkUtilization` case with `if/else if` severity logic; test all threshold levels; observe Type 2 clear behavior
- **4.3.7** *(15 min)* — Create `CUSTOMERS.lookup` file; add `table` declaration and `lookup()` call; verify `@CustomerID` and `@Service` populated
- **4.3.8** *(10 min)* — Add `WirelessAssociation` discard case; confirm event does not reach ObjectServer
- **4.3.9** *(10 min)* — Add `@ExpireTime = 30` to `LinkUtilization`; observe auto-clear after 30 seconds
- **4.3.10** *(5 min)* — Read reference section on additional functions (no tasks)
- **4.3.11** *(10 min)* — Add `%RawCapture = 0` globally; set `%RawCapture = 1` in `default` case only; test selective capture

 Checkpoint: `LinkDown`/`LinkUp` correlation works. `LinkUtilization` severity changes dynamically. `CustomerID` and `Service` fields populated. Discard and expire both working.

1:05 PM — Lunch *(60 min)*

2:05 PM — Section 5.0 & 5.1: SNMP Setup and Overview *(30 min)*

- Create SNMP `Traps` filter in Web GUI (`Manager like 'MTTrapd'`)
- Read SNMP fundamentals: OIDs, MIBs, varbinds, SNMPv1/v2c/v3
- Understand MTTrapd probe role and NcKL architecture

2:35 PM — Section 5.2: Configure MTTrapd with NcKL *(45 min)*

Hands-on — SSH, schema setup, environment variable, properties file, start probe

- Run `advcorr.sql` NcKL schema setup script
- Export and persist `NC_RULES_HOME`; add to probe `.env` file
- Add 5 property lines to `mttrapd.props` (Server, MessageLevel, RulesFile, MessageLog, RawCapture)
- Syntax-check NcKL rules; start `nco_p_mttrapd`
- Verify on port 162; confirm Probe Heartbeat in Web GUI

 Checkpoint: `nco_p_mttrapd` listening on UDP 162. Heartbeat event visible in Web GUI.

3:20 PM — Break *(15 min)*

3:35 PM — Section 5.3: Test with Standard IF-MIB Traps *(30 min)*

Hands-on — SSH, `snmptrap` commands, Web GUI, rawcapture inspection

- Send `linkDown` trap via `snmptrap`; observe event in Web GUI
- Inspect rawcapture tokens; understand varbind naming
- Send `linkUp` trap; observe `generic_clear` auto-resolution

 Checkpoint: `linkDown` appears and clears when `linkUp` is sent.

4:05 PM — Section 5.4: Build and Load a Custom MIB with MIB Manager *(55 min)*

Hands-on — Windows (MIB Manager GUI) + SSH (NcKL integration)

- Save `IBM-TRAINING-MIB.txt` on Windows workstation
- Import into MIB Manager; browse OID tree; confirm 3 trap definitions
- Export as NcKL 3.x format
- SCP exported `ibmTraining` directory to lab VM
- Move files to `$NC_RULES_HOME/include-snmptrap/ibmTraining/`
- Add lookup and rules include lines to `snmptrap.rules` (alphabetical position)
- Syntax check; restart `nco_p_mttrapd`
- Send test `serverDown` trap; confirm OID resolution in rawcapture

 Checkpoint: `serverDown` trap produces a named, readable event. Rawcapture shows `$ibmTrainingServerName` token.

5:00 PM — End of Day 2

- Save all work; leave ObjectServer, JazzSM, and MTTrapd running
- Brief instructor recap

Day 3 (Morning) — NcKL Customization, Process Agent, Cloning, Gateway

8:30 AM — Day 3 Warm-Up *(15 min)*

- Verify all processes running: `nco_pa_status`, `nco_ping EVENTS`, Web GUI login
- Brief review of Day 2 key concepts

8:45 AM — Section 5.5: Customizing the NcKL-Generated Rules *(75 min)*

Hands-on — SSH, user include file and severity lookup file

- **5.5.1** *(10 min)* — Review generated rules file; understand the `switch($enterprise)` → `switch($specific-trap)` flow and the `serverDown` `@Identifier`
- **5.5.2** *(10 min)* — Send two `diskUtilization` traps with different utilization values; observe broken deduplication; understand why
- **5.5.3** *(30 min)* — Edit `user.include.snmptrap.rules`: add `@ExtendedAttr = nvp_add($*)`, set `@Node`, fix `@Identifier`, improve `@Summary`, set `@Severity`, set `@AlertKey`, align `@AlertGroup` for `serverDown`/`serverUp` correlation
- **5.5.4** *(15 min)* — Edit severity lookup file: set `serverDown` to Critical/Type 1, `serverUp` to Indeterminate/Type 2, `diskUtilization` to Major/Type 1/ExpireTime 1800
- **5.5.6** *(10 min)* — Run all four final tests: `serverDown`, `diskUtilization /var`, `diskUtilization /home`, `serverUp` clears `serverDown`

 Checkpoint: Two disk utilization events remain (separate mount points); `serverDown` clears on `serverUp`; summary and severity are human-readable and correct.

10:00 AM — Break *(15 min)*

10:15 AM — Section 6: Managing OMNIbus Processes with Process Agent *(60 min)*

Hands-on — SSH, edit `nco_pa.conf`, `nco_pa_*` commands

- **6.0** *(10 min)* — Inspect current Process Agent state; review existing `nco_pa.conf`
- **6.1** *(20 min)* — Add `SyslogdProbe` and `MttrapdProbe` process definitions; add `Probes` service (`Non-Master`, `ServiceStart = Auto`)
- **6.2** *(10 min)* — Restart `nco_pad`; verify all three processes `RUNNING`
- **6.3** *(10 min)* — Stop and start `SyslogdProbe` individually; confirm isolation
- **6.4** *(10 min)* — Stop and start `Probes` service; confirm ObjectServer unaffected

 Checkpoint: `nco_pa_status` shows `Core` (ObjectServer) and `Probes` (both probes) all `RUNNING`. Individual and service-level control demonstrated.

11:15 AM — Section 7: Cloning an ObjectServer *(60 min)*

Hands-on — SSH, `nco_osreport`, `nco_dbinit`, `omni.dat`, `nco_igen`, Process Agent

- **7.1** *(10 min)* — Export EVENTS with `nco_osreport -dbinit`; confirm 6 SQL files generated
- **7.2** *(5 min)* — Review `system.sql`, `automation.sql`, `security.sql`; understand which file to omit
- **7.3** *(10 min)* — Initialize EVENTS_BACKUP with `nco_dbinit` (5 files, no `alertsdata.sql`)
- **7.4** *(10 min)* — Add EVENTS_BACKUP entry to `$NCHOME/etc/omni.dat` (port 4101); run `nco_igen`
- **7.5** *(10 min)* — Add `BackupObjectServer` process and `BackupObjectServerService` (`Non-Auto`) to `nco_pa.conf`
- **7.6** *(15 min)* — Restart Process Agent; start `BackupObjectServerService`; verify `nco_ping EVENTS_BACKUP`; confirm schema and zero alert count via `nco_sql`

 Checkpoint: Both EVENTS and EVENTS_BACKUP running independently. Clone has correct schema, no alert data.

12:15 PM — Lunch *(45 min)*

1:00 PM — Section 8: Configuring a Bidirectional ObjectServer Gateway *(90 min)*

Hands-on — SSH, gateway properties, map file, `omni.dat`, Process Agent, bidirectional SQL tests

- **8.1** *(10 min)* — Diff `alerts.status` schemas; confirm `CustomerID` and `CircuitID` exist on both
- **8.2** *(10 min)* — Create `$OMNIHOME/gates/events_backup/`; copy and restrict template files
- **8.3** *(10 min)* — Add 9 property lines to `objserv_bi.props` (gateway name, both ObjectServer connections, map and startup file paths)
- **8.4** *(10 min)* — Add `CustomerID` and `CircuitID` mappings to `objserv_bi.map` (no trailing comma)
- **8.5** *(15 min)* — Add EVENTS_GATE to `omni.dat`; run `nco_igen`; start gateway manually; verify startup banner; test replication with `LinkDown` event; stop manual gateway with `pkill`
- **8.6** *(15 min)* — Add gateway process to `nco_pa.conf`; add to `BackupObjectServerService` with `BackupObjectServer` dependency; restart Process Agent; start service; verify all processes stable
- **8.7** *(15 min)* — Test A: insert from EVENTS, verify in EVENTS_BACKUP, update `CustomerID`, verify update replicated; Test B: insert from EVENTS_BACKUP, verify in EVENTS
- **8.8** *(5 min)* — Delete test events; confirm deletion replicates to both sides

 Checkpoint: Inserts, updates, and deletes all replicate bidirectionally. `CustomerID` custom field replicates correctly. All processes stable in `nco_pa_status`.

2:30 PM — Course Wrap-Up and Review *(30 min)*

- Instructor-led review of the full pipeline: source → probe → ObjectServer → operator → Process Agent → clone → gateway
- Open Q&A
- What to expect in the Netcool/Impact training that follows

3:00 PM — OMNIbus Lab Course Complete

Quick Reference — What You've Built

By the end of Day 3, your lab environment contains:

Component	Name	Status
ObjectServer (primary)	`EVENTS`	Running, managed by Process Agent
ObjectServer (clone)	`EVENTS_BACKUP`	Running on-demand, managed by Process Agent
Bidirectional Gateway	`EVENTS_GATE`	Running, replicating between both ObjectServers
Syslogd probe	`SyslogdProbe`	Running with custom training rules, managed by Process Agent
MTTrapd probe	`MttrapdProbe`	Running with NcKL + IBM-TRAINING-MIB rules, managed by Process Agent
Web GUI	JazzSM / `ncoadmin`	Running on port `16311`

Quick Reference — Key Files Created

File	Purpose
`$OMNIHOME/probes/linux2x86/syslogd-training.rules`	Custom syslogd probe rules
`$OMNIHOME/probes/linux2x86/CUSTOMERS.lookup`	Lookup table mapping routers to customers
`$NC_RULES_HOME/include-snmptrap/ibmTraining/`	Custom MIB rules, lookup, and user include files
`$NCHOME/etc/omni.dat`	Communications endpoints for EVENTS, EVENTS_BACKUP, EVENTS_GATE
`$OMNIHOME/etc/nco_pa.conf`	Process Agent configuration for all managed processes
`$OMNIHOME/gates/events_backup/objserv_bi.props`	Gateway properties
`$OMNIHOME/gates/events_backup/objserv_bi.map`	Gateway field mapping including custom columns